CIS 5810 FINAL PROJECT · GROUP [#]

Magic Eraser

Type what to remove from a video. It disappears.



Input



“remove the person”

Daniel Zhang · [member 2] · [member 3]

github.com/Norrozo/5810finalproject

THE PROBLEM

Removing an object from video takes hours of frame-by-frame editing

What makes it hard

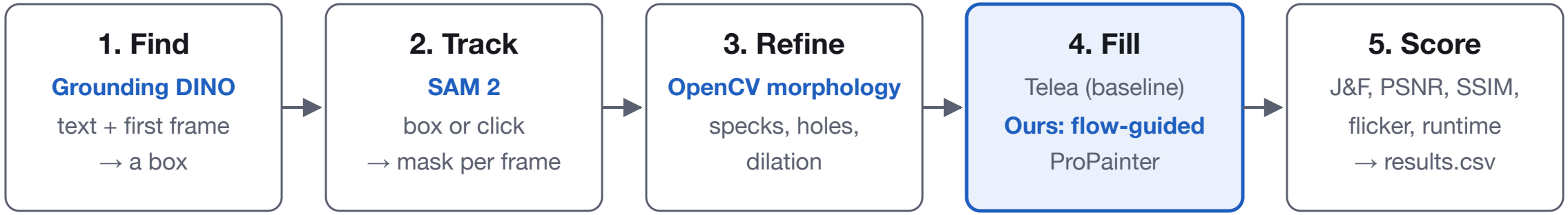
- **Find** the object from a plain description
- **Track** it through every frame as it moves and turns
- **Fill** what was behind it, which the camera may never have seen
- Do it **consistently**, so the fill doesn't shimmer frame to frame

Our goal

- Input: a 5–10 s clip + text (“remove the dog”)
- Output: the same clip, object gone, no blur or flicker
- Audience: casual creators removing a photobomber without After Effects
- Measure every change against a fixed baseline

SYSTEM DESIGN

Five stages, each a separate script that reads and writes files



Swap one stage, keep the rest fixed → every iteration changes exactly one thing.

Why files between stages: a crashed run resumes where it stopped, and later versions reuse the baseline's masks, so fill methods are compared on identical masks.

Same code everywhere: the benchmark, the Colab notebook and the web demo all call these five stages.

SAM 2: Meta 2024 · Grounding DINO: IDEA 2023 · ProPainter: ICCV 2023

HOW WE MEASURE

Real videos have no answer key, so we build one



Input: a DAVIS dog pasted onto another video



Tracking truth: the exact paste mask



Fill truth: the untouched video

- **Test set:** 10 clips from DAVIS *val*, frozen before any results
- **Tuning set:** 5 clips from DAVIS *train*. Settings are chosen here, never on the test set

- **J&F** ↑ did we track the right pixels?
- **PSNR mask** ↑ is the filled hole close to the truth? (dB)
- **Flicker** ↓ does the fill shimmer? (clean video: 7.6)

DAVIS 2017, 480p, 40–60 frames per clip

BASELINE (B0): CLICK → SAM 2 → OPENCV TELEA

Tracking is nearly perfect; the fill is a fog



Input



B0 result

Telea fills the hole by smudging the colors at its edge inward. It never looks at other frames, so a big hole becomes a smear, and the smear changes every frame.

0.943

J&F: tracking is nearly solved

16.4 dB

PSNR inside the hole: visibly wrong

2.5×

the clean video's flicker

Means over 10 test clips · 1.38 s/frame, 97% of it SAM 2

ITERATIONS I1 AND I2

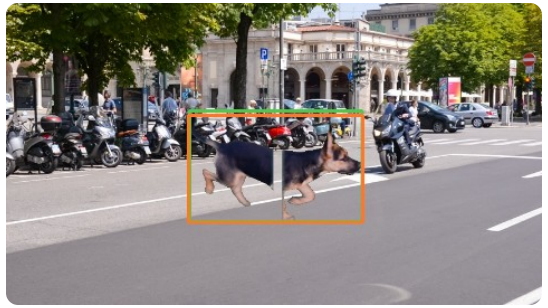
Bigger masks hurt; text prompts work 9 of 10 times

I1 Grow the mask 5 px

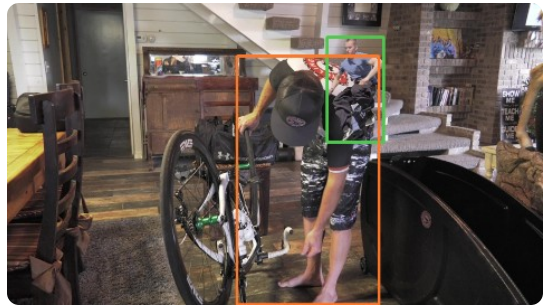
PSNR mask 16.39 → **15.51**. Pasted objects have razor-sharp edges, so there's no fuzzy halo to clean up. A bigger mask is just a bigger hole to blur.

A negative result that exposes a limit of our benchmark. On a real video (a bear's fur) a halo does appear, so we'll re-test on real clips.

I2 Type the object's name



“dog”: one box covers both halves. J&F 0.60 → 0.95



“person”: picked the wrong person. J&F 0.11

■ true object ■ Grounding DINO's box

Grounding hit = box IoU > 0.5 with the true object

ITERATION I3: OUR OWN METHOD

Flow-guided fill: copy the real background from other frames

1. **Optical flow** (Farnebäck): where did each pixel move between neighboring frames?
2. **Complete the flow**: inside the hole the flow follows the dog, so replace it with the smooth background motion around it.
3. **Propagate**: follow the flow to a neighbor frame; if the background is visible there, copy it. Sweep forward and backward.
4. **Fallback**: whatever no frame ever saw goes to Telea.



B0: fog

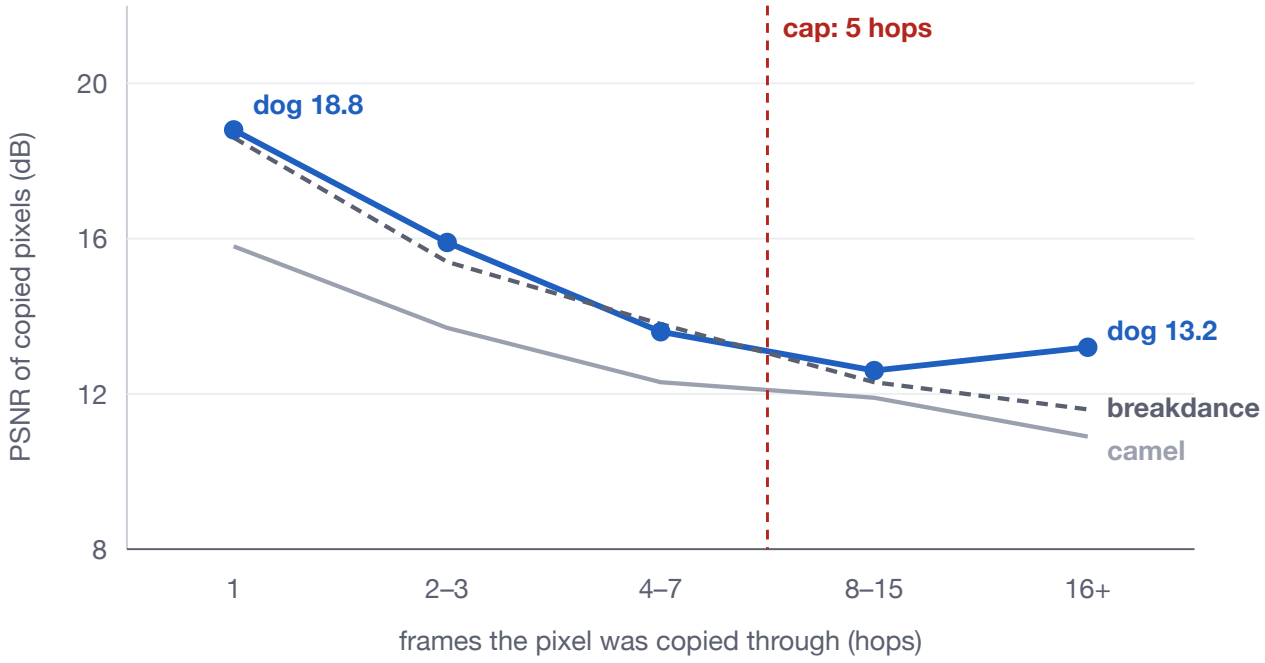


Ours: real cart recovered

Flicker **–13%**, because the same real background is copied into every frame. But PSNR *dropped* at first (16.39 → 15.98). Why?

ITERATION I3B: DEBUGGING OUR METHOD

Each extra hop costs accuracy, so we cap hops at 5



Each hop adds a little optical-flow error and a little interpolation blur. After 8+ hops, copied pixels are **worse than Telea**.

We chose the cap on the **tuning set** (5 → 20.65 dB vs 19.46 uncapped), then ran it once on the test set.

	PSNR mask	Flicker
B0	16.39	18.75
I3b (ours)	**16.50**	**16.31**

Hop analysis on B0 masks; chart shows 3 test clips

ITERATION I4: PROPAINTER

A learned video inpainter removes the fog



Input | B0 Telea | I3b ours | **I4 ProPainter** | Truth

Same idea as ours, every step learned: RAFT flow, a network that completes flow, propagation in image and feature space, and a **transformer that invents** background no frame ever showed. That last step is exactly what our method lacks.

+4.75
dB PSNR mask vs B0

–51%
flicker vs B0

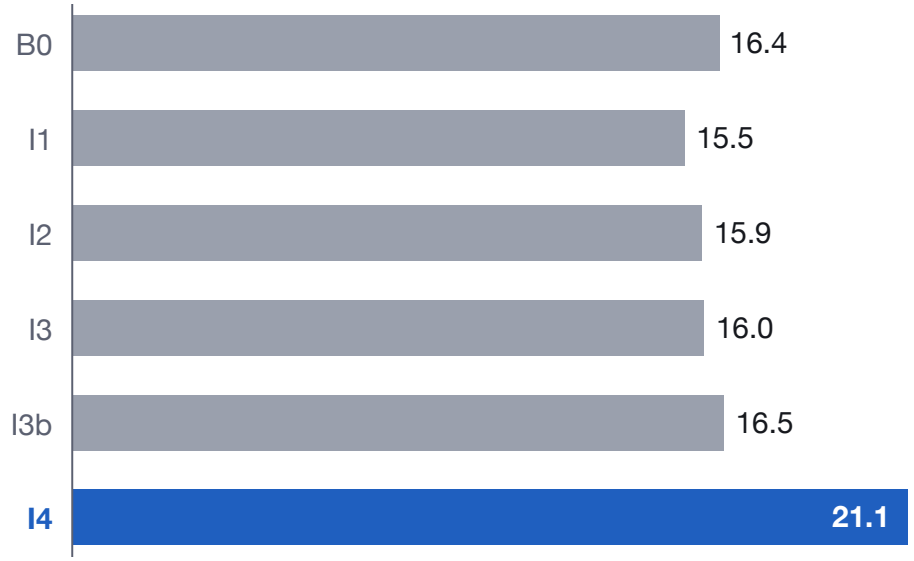
Better on all 10 clips. Cost: about 4× runtime.

Zhou et al., ProPainter, ICCV 2023 · run in its own Python environment

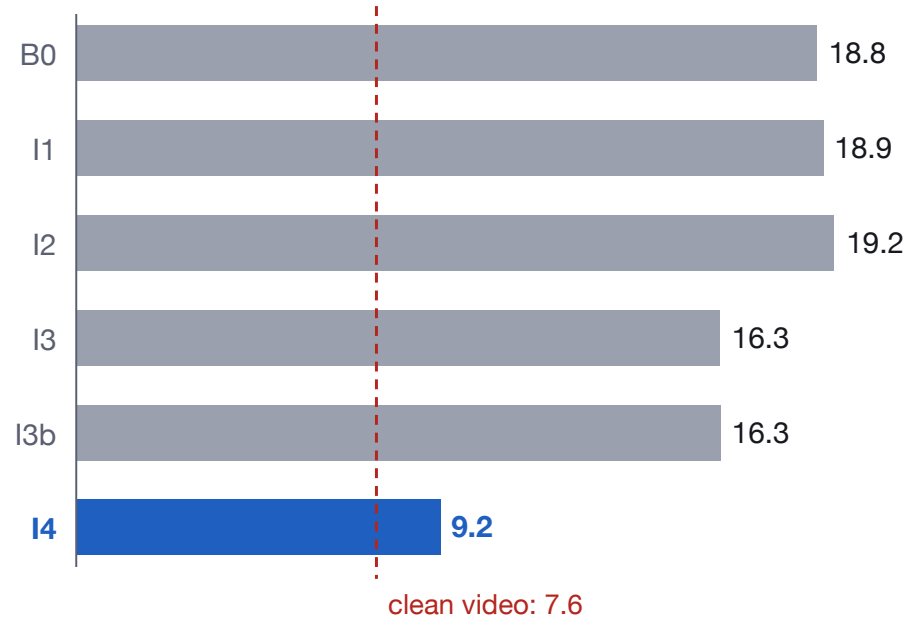
RESULTS SO FAR

ProPainter cut the error and halved the flicker

PSNR inside the hole (dB) ↑



Flicker ↓



Means over the 10 test clips. Tracking (J&F 0.943) is identical for all fill methods because they share the baseline's masks; I2 is 0.890 due to the one grounding miss.

THE RESULTS IN PLAIN WORDS

Five numbers anyone can read

Version	Mask accuracy	Fill score	Steadiness	Speed
Baseline: click, OpenCV fill	94%	24/100	2.8×	1.4 s/frame
Our flow-guided fill (I3b)	94%	26/100	2.5×	1.5 s/frame
ProPainter fill (I4)	**94%**	**63/100**	**1.1×**	**5.4 s/frame**
ProPainter with a perfect mask	100%	68/100	1.0×	–

Real videos (10 clips)	Right object	Mask accuracy
Click, whole object (I8)	clicked	86%
Text, asks “which one?” if unsure (I2c)	7/10	67%

Mask accuracy: overlap with the true object (J&F).
Fill score: 0 = object still there, 100 = identical to the real background (from LPIPS).

Steadiness: flicker vs. the real video; 1.0× = as steady as real footage.
Speed: on a MacBook (M5). Top table: pasted benchmark, 10 clips (the true background is known).

REAL VIDEOS: TEXT PROMPT → SAM 2 → PROPAINTER

On real clips it works when the background moves past the object



“remove the ball”



clean



“remove the person”



clean, faint shadow



“remove the tennis player”



racket left floating



“remove the bear”

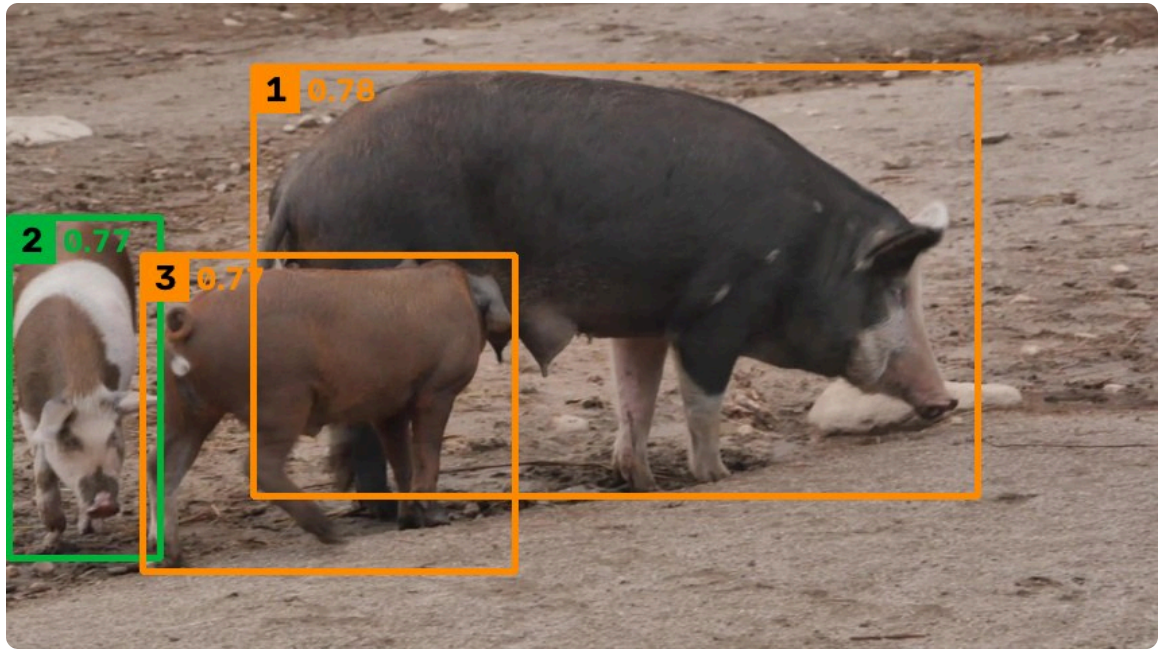


smudge: camera follows the bear

DAVIS clips not used in the benchmark

REAL-VIDEO TEST SET · ITERATIONS I2B, I2C

On real video, the weak stage is *which* object; so we ask when unsure



“piglet”: the top box (0.78) is the adult pig. Three scores within 0.01, so I2c asks; the user clicks #2.

10 real DAVIS clips	Right object	J&F
B0 click	–	0.798
I2 text	5/10	0.482
I2b + CLIP re-rank	5/10	0.483
I2c **ask if unsure**	**7/10**	**0.674**

Designed on a separate real **dev** set (14 clips): the right object was in the top 3 on 14/14, and every miss had a close runner-up. Rule: ask if runner-up ≥ 0.7 × best → asks on 3 of 10 test clips. CLIP re-ranking scored 6/14 on dev: rejected.

When grounding is right, tracking real video is fine: J&F 0.91.

Benchmark: a simulated user clicks the true object · results/real_results.csv

Re-checking the tracker with the detector made it worse, and we know why

Idea. Run the detector again every 10 frames; if SAM 2's mask matches no detection, assume it drifted and restart it on the detection.

I5 trusted only confident detections. On real *loading* the right man scored 0.32 (< 0.35) on frame 10, so the correct track was "fixed" onto another man: J&F 0.99 → 0.24.

I5b confirms the track against every candidate. Checked on the dev set first: fewer false alarms, still worse. Every remaining "fix" hit a frame where the detector didn't see the object at all.

Real J&F	Dev (14)	Test (10)
I2c, no re-detection	**0.912**	**0.674**
I5	0.799	0.600
I5b	0.866	not run

0 of 24

real clips where re-detection made a correct fix. SAM 2 never lost the object; the detector missed it in single frames.

Decision: rejected. The detector is the less reliable signal, so it must not override the tracker.

results/real_dev_results.csv · real_results.csv · I5b rejected on dev, so never run on test

ITERATIONS I6, I8: TRACKING FROM A CLICK

One click is ambiguous: the glasses, or the man?



B0: SAM 2 keeps the mask it is most sure of: the sunglasses



I8: keep the largest of SAM 2's three masks: the whole man

Mask accuracy (J&F)	Real dev	Real test	Pasted
B0	0.791	0.798	0.943
I8 **largest mask**	**0.903**	**0.858**	**0.941**

Fixed 4 part-vs-whole clips (e.g. 0.08 → 0.90). **Broke** 2 where DAVIS labels only part of the thing (a rider without his bike): for removal, the whole is usually what you want.

I6 A smaller SAM 2 was ≤ 1.7× faster but lost up to 0.19 J&F on dev: rejected.

Rule chosen on dev frame 0 · demo now uses I8 for clicks

A BETTER EVALUATION FRAMEWORK

We score each stage on its own, so one stage's errors can't hide in another

1 · Grounding

Did the text pick the right object? Frame 0, box IoU > 0.5.

7/10

real clips (I2c) · 9/10 synthetic

2 · Tracking

Masks from a correct start: a click, or text clips grounded right.

0.80–0.91

real J&F · 0.94 synthetic

3 · Fill

Every fill method gets the **true** masks, so only the fill is measured.

0.125

LPIPS, ProPainter (Telea 0.249) · flicker = clean video's

4 · End to end

The full pipeline: what a user gets. All stages' errors combined.

0.142

LPIPS, I4 · perfect masks would add +1.1 dB PSNR

What the split tells us. On real video, text versions lose J&F 0.91 → 0.67 to grounding but almost nothing to tracking; ProPainter's fill is already as steady as the real video. The next gains are in *what* to remove, not how to track or fill.

How: `--overlay configs/fill_only.yaml` swaps SAM 2 for the true masks; `python -m src.stage_report` builds the four tables from the result CSVs (results/stage_report.md). Fill ranking is the same with true masks as end to end, so earlier fill comparisons were fair.

EVALUATING THE EVALUATION

Do the metrics measure what we claim? We tested them on fakes

Fake output	PSNR mask ↑	LPIPS mask ↓	Flicker ↓
Perfect (the clean video)	100	0.000	7.3
Object not removed	10.4	0.404	36.3
Flat gray box	12.0	0.439	**9.7**
Blurred background	**20.5**	0.305	**5.9**
Sharp background, 4 px off	17.7	**0.087**	10.1
Right pixels, flickering	37.3	0.007	**11.6**

Mean over the 10 synthetic test clips, scored by the same code as every result. Flicker = RAFT-aligned.

- **PSNR rewards blur** (20.5 > 17.7); LPIPS ranks them the way people do. → LPIPS is primary.
- **Flicker can't tell a gray box from a good fill**; blur is even steadier than the real video. → Read flicker only next to LPIPS.
- **Only flicker catches flickering**; per-frame scores call it near-perfect. → A temporal metric is required.
- Farneback vs RAFT flicker rank outputs the same (ρ = 0.98): our method isn't favored.

python -m src.metric_check · results/metric_check.csv, metric_agreement.csv

PROOF OF ITERATION

Every change was one change, measured, then kept or rejected

Tag	One change	Stage	Measured on	Result	Decision
B0	Click → SAM 2 → Telea	all	synthetic + real	J&F 0.943 · LPIPS 0.303	baseline
I1	Grow mask 5 px	refine	synthetic	PSNR mask 16.39 → 15.51	rejected
I2	Text prompt (Grounding DINO)	grounding	synthetic + real	right object 9/10 · real 5/10	kept
I3	Our flow-guided fill	fill	synthetic	flicker –8% (RAFT) · PSNR mask down	analysed
I3b	Cap at 5 hops (tuned on dev)	fill	dev → synthetic	LPIPS 0.294 · PSNR mask 16.50	kept: fast
I4	ProPainter	fill	synthetic	LPIPS 0.142 · flicker 19.0 → 8.5	kept: best
I2b	CLIP re-ranking	grounding	real dev	11/14 → 6/14	rejected
I2c	Ask “which one?” when unsure	grounding	dev → real test	5/10 → 7/10 · J&F 0.48 → 0.67	kept
I5	Re-detect every 10 frames	tracking	real test	J&F 0.674 → 0.600	rejected
I5b	Confirm vs. all candidates	tracking	real dev	J&F 0.912 → 0.866	rejected
I6	Smaller SAM 2 (speed)	tracking	real dev	≤ 1.7× faster · J&F 0.791 → ≤ 0.710	rejected
I8	Click selects whole object	tracking	dev → real test	J&F 0.798 → 0.858 (real)	kept

Settings chosen on dev sets (DAVIS train), then run once on the test sets (DAVIS val). Every row: own config, own outputs, a log entry.

The remaining failures are about *what* to remove, not how to fill

Lessons

- Measure before fixing: the hop analysis turned a failed iteration into a fix
- Tune on a separate set, or the test numbers lie
- Score each stage alone: on real video the loss is grounding, not tracking or filling
- Test the metrics too: PSNR rewards blur, flicker rewards a gray box
- A pasted benchmark hides real effects: look-alikes, soft edges, shadows

Next iterations

When	Iteration
Nov 20	Let the user pick among SAM 2's masks for a click (like I2c); faster fill
Dec 6	**I7** multiple objects ("the player and the racket"), shadows; failure study
Dec 10	Final report from the experiment log

LIVE DEMO

Upload a video. Type what to remove.

Gradio app on the same five stages · `python -m src.demo` · Colab: `notebooks/colab.ipynb`



Input



“remove the person”